ASSIGNMENT ON PRIMITIVE DATA STRUCTURES, NESTED RECURSION, AND ARRAYS

Subject: Data Structures and Algorithms
Topic: Primitive Data Structures, Nested Recursion, and Scientific Computing
Submitted by: DINESH KARTHICK R
Register Number: 192524121
Department: AI & Data Science

1. DEFINE PRIMITIVE DATA STRUCTURES WITH SUITABLE EXAMPLES

1.1 Introduction

Data structures are ways of organizing and storing data in a computer so that it can be accessed and processed efficiently. They are broadly classified into primitive and non-primitive data structures. Primitive data structures are the basic data types provided by a programming language and are used to represent single values.

1.2 Definition of Primitive Data Structures

Primitive data structures are fundamental data types that directly store a simple value, such as an integer, character, or logical value. They form the building blocks for more complex structures such as arrays, linked lists, stacks, queues, and trees.

The exact primitive types available depend on the programming language. Common examples include integer, floating-point, character, and Boolean types.

1.3 Types of Primitive Data Structures

A. Integer

An integer data type stores whole numbers without a fractional part. It can represent positive numbers, negative numbers, and zero, subject to the range supported by the language and the chosen integer size.

Example: `int age = 18;` stores the whole number 18.

B. Floating-Point

A floating-point data type stores numbers that contain a fractional or decimal part. It is useful when calculations require values such as measurements, averages, or scientific quantities.

Example: `float temperature = 36.5;` stores a decimal value.

C. Character

A character data type stores a single character, such as a letter, digit, or symbol. In many languages, a character is represented using a character encoding.

Example: `char grade = 'A';` stores the character A.

D. Boolean

A Boolean data type represents one of two logical values: true or false. It is commonly used in conditions, comparisons, and decision-making statements.

Example: `bool isPassed = true;` indicates that the condition is true.

E. Double-Precision Floating-Point

The double data type stores floating-point values with greater precision than a typical float type. It is useful in calculations where additional precision is required.

Example: `double pi = 3.1415926535;` stores an approximation of pi.

1.4 Summary of Primitive Data Types

Data type	Purpose	Example
Integer	Stores whole numbers	int count = 25;
Float	Stores decimal values	float rate = 2.5;
Character	Stores one character	char grade = 'A';
Boolean	Stores true or false	bool valid = true;
Double	Stores decimal values with greater precision	double pi = 3.14159;

1.5 Characteristics of Primitive Data Structures

1. They represent simple and individual values.
2. They are supported directly by programming languages.
3. Their storage size and value range depend on the language and implementation.
4. They are used to build more complex data structures.
5. They support basic operations such as assignment, arithmetic, comparison, and logical testing.

1.6 Applications

Primitive data types are used throughout programming. Integers can count students or store item quantities; floating-point types can represent height, temperature, and speed; characters can store grades or symbols; and Boolean values can represent whether a student passed or whether a condition is satisfied.

1.7 Conclusion

Primitive data structures are the basic building blocks of data representation in a program. Integer, float, double, character, and Boolean types represent different kinds of simple values. Selecting a suitable primitive type helps a program store data correctly and perform operations efficiently.

2. ANALYZE NESTED RECURSION WITH RECURRENCE RELATIONS

2.1 Introduction

Recursion is a programming technique in which a function calls itself to solve a smaller instance of a problem. In nested recursion, a recursive function uses the result of one recursive call as the argument of another recursive call. This makes the evaluation more complicated than simple recursion because the next call may depend on the result of an earlier call.

2.2 Definition of Nested Recursion

Nested recursion occurs when a recursive call is placed inside the argument of another recursive call. The function cannot complete the outer call until the inner call has returned a value.

A common example is the McCarthy 91 function, which is defined using nested recursion and is useful for illustrating how a recurrence-like definition controls recursive evaluation.

2.3 Example: McCarthy 91 Function

The McCarthy 91 function M(n) is defined as follows:

```
M(n) = n - 10,                          if n > 100\n       = M(M(n + 11)),
```

For inputs greater than 100, the function returns n – 10 directly. For inputs of 100 or less, it makes a nested recursive call. First, M(n + 11) is evaluated; its returned value then becomes the argument of the second call M(...).

2.4 Step-by-Step Evaluation

Consider the input n = 99. Since 99 is not greater than 100, the second rule is applied:

```
M(99) = M(M(110))
```

The inner call has an argument of 110, which is greater than 100:

```
M(110) = 110 - 10 = 100
```

The value returned by the inner call becomes the argument of the outer call:

```
M(99) = M(100)
```

Since 100 is not greater than 100, the recursive rule applies again:

```
M(100) = M(M(111))\nM(111) = 111 - 10 = 101\nM(100) = M(101) = 101 - 10 = 91
```

Therefore, M(99) = 91. This example shows that nested recursion can create a sequence of dependent calls before returning the final result.

2.5 Understanding Recurrence Relations

A recurrence relation describes a quantity in terms of one or more smaller or earlier instances of the same quantity. In algorithm analysis, recurrence relations are used to describe running time or the number of operations performed by a recursive algorithm.

For a simple recursive algorithm that reduces the input by one and performs constant work per call, the running time may be written as:

```
T(n) = T(n - 1) + c,     for n > 1\nT(1) = d
```

Here, c and d are constants. Expanding the relation gives T(n) = T(n – 2) + 2c, then T(n) = T(n – 3) + 3c, and so on. After approximately n steps, T(n) = T(1) + (n – 1)c. Therefore, the time complexity is O(n).

2.6 Recurrence Relation for a Divide-and-Conquer Algorithm

When an algorithm divides a problem into two subproblems of half the size and spends linear time combining their results, its recurrence relation can be written as:

```
T(n) = 2T(n / 2) + cn,     for n > 1\nT(1) = d
```

At each level, the total combining work is proportional to n. The number of levels is approximately log■ n because the problem size is halved at each level. Thus, the total work is proportional to n log n, giving a time complexity of O(n log n), as in merge sort.

2.7 Analyzing Nested Recursion

Nested recursion is often harder to analyze than a simple recurrence because the argument of a recursive call depends on the result of another call. A single equation such as T(n) = T(n – 1) + c may not accurately describe the number of calls.

A practical analysis should identify the base condition, trace the order of inner and outer calls, determine how the argument changes, and count the total calls or operations. If the function has several possible branches, each branch must be considered. A recurrence relation should only be stated after the actual recursive behavior is understood.

2.8 Advantages and Limitations

Advantages

- It expresses problems with dependent or repeated evaluations in a compact way.
- It is useful for mathematical definitions and some parsing or evaluation problems.
- It helps demonstrate how one recursive result can control another recursive call.

Limitations

- The order of evaluation can be difficult to trace.
- Many nested calls can consume considerable call-stack memory.

- Incorrect base cases may cause excessive recursion or non-termination.
- Time complexity can be difficult to derive without a careful trace or a suitable mathematical model.

2.9 Conclusion

Nested recursion occurs when one recursive call is used inside another call's argument. The McCarthy 91 function illustrates this behavior, while recurrence relations provide a mathematical method for analyzing recursive algorithms. Accurate analysis requires tracing the base case, recursive calls, argument changes, and work performed at each step.

3. EXPLAIN USE OF ARRAYS IN SCIENTIFIC COMPUTING

3.1 Introduction

Scientific computing uses computers to solve scientific, engineering, and mathematical problems. These problems often involve large collections of numerical data, including measurements, experimental results, simulation values, and mathematical matrices. Arrays are important in scientific computing because they store many values of the same type in an organized form.

3.2 Definition of an Array

An array is a linear data structure that stores a fixed or specified number of elements of the same data type in indexed positions. Each element can be accessed using its index. One-dimensional arrays represent lists, while two-dimensional and multidimensional arrays can represent tables, matrices, images, and scientific grids.

Example:

```
temperature = [28.5, 29.0, 30.2, 29.8, 28.9]
```

This array stores five temperature measurements. Each value can be accessed using its index, allowing a program to process the readings one by one.

3.3 Uses of Arrays in Scientific Computing

A. Storing Experimental Data

Arrays store observations collected from experiments, sensors, or laboratory instruments. For example, a temperature array can hold hourly readings, making it easy to calculate the average, maximum, minimum, or change over time.

B. Matrix and Vector Operations

Vectors and matrices are central to linear algebra, numerical analysis, and engineering. Two-dimensional arrays represent matrices, while one-dimensional arrays can represent vectors. Arrays support operations such as addition, multiplication, transposition, and solving systems of equations.

C. Numerical Simulation

Scientific simulations calculate the behavior of systems over time or space. Arrays can store values such as pressure, velocity, density, and temperature at different grid points. In weather modelling, for example, a multidimensional array can represent measurements across locations and atmospheric layers.

D. Image and Signal Processing

A digital image can be represented as a two-dimensional array of pixel values, or a three-dimensional array for colour channels. Similarly, sampled audio signals can be stored in arrays and processed using filtering, transformations, and statistical calculations.

E. Statistical Analysis

Arrays make it convenient to calculate totals, means, variance, standard deviation, and other statistical measures across large datasets. They also allow repeated operations to be applied systematically to many observations.

F. Efficient Numerical Computation

Array-based libraries can perform operations on many elements efficiently. Contiguous storage and optimized numerical routines can improve memory access and execution speed, especially when working with large datasets.

3.4 Example: Calculating Average Temperature

Consider five temperature readings stored in an array. The average is calculated by adding all values and dividing the total by the number of readings.

```
ALGORITHM AverageTemperature(A, n)\n  total = 0\n  FOR i = 0 TO n - 1\n      tota
```

For the array [28.5, 29.0, 30.2, 29.8, 28.9], the total is 146.4. Dividing by 5 gives an average temperature of 29.28. This example shows how arrays allow a collection of measurements to be processed using a simple loop.

3.5 Advantages of Arrays in Scientific Computing

1. Direct access: An element can be accessed by its index in constant time in a typical array implementation.

2. Organized storage: Related numerical values are stored in a structured sequence or grid.

3. Efficient processing: Loops and vectorized operations can process many values systematically.

4. Convenient mathematical representation: Arrays naturally represent vectors, matrices, images, and simulation grids.

5. Compatibility with scientific libraries: Numerical libraries commonly use arrays as their main data representation.

3.6 Limitations of Arrays

1. A traditional array may have a fixed size, so changing the number of elements can require additional work.

2. In many languages, an array stores elements of one type, which limits mixed-type storage.

3. Large multidimensional arrays can consume substantial memory.

4. Incorrect index values can cause errors or access invalid positions in languages that do not check bounds automatically.

3.7 Conclusion

Arrays are essential in scientific computing because they provide an organized and efficient way to store and process numerical data. They are used for experimental measurements, vectors and matrices, simulations, image processing, and statistical analysis. Although large arrays require careful memory and index management, they remain one of the most useful structures for scientific and engineering applications.